

Design Principles for Scientific Software

September 30, 2026

Contents

About these guidelines	2
Normative language	2
Terminology	2
1 Guiding priorities	3
1.1 Build the right thing before building the thing right	3
1.2 Get your priorities right	3
1.3 Pay as you grow	3
1.4 KISS: Keep it simple, stupid	4
2 Scope	4
2.1 Own only the core domain	4
2.2 Code is for life, not just for Christmas	4
2.3 Don't call us, we'll call you	5
2.4 Operate on data objects, not files	5
2.5 Avoid plotting in core functionality	5
3 Code structure and abstraction	5
3.1 Derive abstractions from applications, not applications from abstractions	6
3.2 Design modules around clear responsibilities	6
3.3 DRY: Don't repeat yourself	6
3.4 No magic numbers	6
3.5 Duplicate code is cheaper than the wrong abstraction	7
4 API design	7
4.1 Write the API first	7
4.2 Make the common case easy, and the uncommon case possible	7
4.3 Name axes, units, and conventions	8
4.4 Numerics and defaults are part of the API	8
4.5 Design stable APIs	8
4.6 Document the method, not the implementation	8
5 Explicit behavior and no side effects	9
5.1 Do what it says on the tin	9
5.2 Don't guess	9
5.3 Do not mutate caller data	9
5.4 Do not touch global state	9
5.5 The library follows the caller's data types	10
5.6 No surprise cost	10
5.7 Importing the package has no side effects	10
5.8 Raise, warn, or stay silent	10
6 Input validation and errors	11

6.1	Garbage in, useful error out	11
6.2	Keep validation proportionate	11
6.3	Missing values and NaN are explicit	11
7	Numerical correctness	12
7.1	Prefer numerically stable formulations	12
7.2	Report conditioning and convergence	12
8	Testing and reproducibility	12
8.1	Testing is not optional	12
8.2	Test invariants, not only examples	12
8.3	Size matters	13
8.4	Reproducibility: the same answer under stated conditions	13
8.5	Compare floats with tolerance	13
8.6	Flaky tests cry wolf	13
8.7	Make every result traceable to a version	13

About these guidelines

These guidelines apply wherever science is turned into software that several people use over and over, whether it is published openly or kept within an organization, and independent of programming language and scientific field. Usually the science is known; the task is to make it reliable, automated, and fit to run at scale. One-off analysis scripts are out of scope.

Normative language

The guidelines distinguish requirements from defaults. The following words carry a fixed meaning throughout, in the spirit of RFC 2119:

must, must not A requirement. Code that violates it is a defect and is fixed, not justified.

should, should not A strong default. A deviation is acceptable only for a concrete reason, which is stated where the deviation occurs, for example in a code comment or in the review. The role of the software (Subsection 1.3) is such a reason; it is stated once for the whole package, for example in its documentation, rather than at each deviation.

may Permitted, but not required.

Plain imperatives such as “prefer” and “avoid” carry the weight of *should*. When two rules pull in different directions, the priorities in Subsection 1.2 decide.

Terminology

Package The software these guidelines apply to, whatever its form: a library, a command-line tool, or a workflow.

Library code The part of a package that other code calls: the scientific logic and everything it needs. Most rules in these guidelines are about library code.

Application layer The part of a tool or workflow that owns the program: command-line entry points, workflow definitions, and job scripts. Rules that restrict library code, such as those on printing, files, and global state, do not apply to it; instead, it has to stay thin (Subsection 2.3).

Host stack The software ecosystem the user already works in: the programming language, its standard array and data-frame types, its established numerical, statistical, and plotting libraries, and the user’s own workflow tools. Examples are the scientific Python ecosystem, R with its

package repositories, and Julia with its package ecosystem. The host stack sets the conventions for data types, randomness, warnings, logging, and packaging that the package follows.

User-facing function Any function, class, or method that is documented for users, including low-level primitives. Together, the user-facing functions form the public API; everything else is internal.

Caller The code that calls a user-facing function, and by extension the user who wrote it.

1 Guiding priorities

1.1 Build the right thing before building the thing right

Establish that the overall approach is correct before investing in optimization, testing, documentation, or polish. Effort spent in the wrong direction is not partially recovered; it is lost in full, and the better the execution, the greater the loss.

Incremental improvement converges on the nearest local optimum, not on the best achievable one. A design can therefore become steadily better while remaining fundamentally misplaced, and no amount of further refinement will reveal this, because refinement measures progress against the current approach rather than against the alternatives to it.

Quality and speed multiply direction rather than substitute for it. Executing a wrong approach quickly and well only arrives sooner at a place that must be abandoned. Refinement also entrenches: the more effort a design has absorbed, the harder it becomes to question, so the assessment should happen early and be repeated deliberately.

Identify which decisions are reversible and which are not. Reversible decisions can be made quickly and corrected later. Irreversible decisions, such as the core abstraction and the public interface, deserve the effort that would otherwise be spent perfecting details, because they determine whether that effort is worth anything at all.

1.2 Get your priorities right

When goals conflict, rank them as follows, and let the higher one win:

1. **Correctness.** The results are right and reproducible; a result that cannot be reproduced cannot be checked (Subsection 8.4).
2. **Clarity.** Code, interface, and behavior can be understood. Code that cannot be understood hides its errors.
3. **Testability.** Correctness can be demonstrated, not merely claimed.
4. **Maintainability.** The package stays correct and clear as it changes.
5. **Performance.** The package is fast enough for the data sizes of real applications (Subsection 8.3).
6. **Features.** More functionality, options, and code are a bonus, never a substitute for the priorities above.

The same ranking resolves conflicts between the rules in these guidelines: when two rules pull in different directions, the rule that serves the higher priority wins. Prefer a small amount of reliable functionality over a large amount of unreliable functionality; code that is produced quickly but is hard to understand, test, or trust only adds to the burden of the package.

1.3 Pay as you grow

Engineering effort has to pay for itself. Not all software in scope deserves the same investment: a tool that three colleagues run on small datasets needs less than a workflow that processes every sample of

a core facility, or a library whose results enter publications and regulatory decisions. Scale the effort with the role of the software, judged by how many people use it, how often and for how long, on how much data, and what a wrong result would cost.

The requirements marked *must* are the floor for all software in scope; most of them cost little and prevent wrong results. What scales with the role is everything above that floor: how thoroughly the *shoulds* are followed, the depth of testing and documentation, the effort put into performance, and the length of the stability commitment.

Roles change, and software usually grows into a larger role without anyone deciding it. Reassess the role whenever the users, the data, or the stakes change, and raise the effort before new users rely on the software, not after the first wrong result.

1.4 KISS: Keep it simple, stupid

Prefer the simplest design that solves the problem well. Simplicity is measured by how much a reader has to hold in mind to understand the code and its behavior: the number of concepts, options, code paths, and special cases, not the number of lines.

Every option multiplies the behaviors that have to be documented and tested, because options interact. Every layer of indirection is a place where a reader has to stop and look elsewhere. Add either only when a concrete application demands it (Subsection 3.1).

Prefer plain, direct code over clever code. When two designs solve the problem equally well, choose the one with fewer concepts. Simple does not mean simplistic: a design that looks simple because it ignores part of the problem, such as missing values or edge cases, is not simple but incomplete.

2 Scope

2.1 Own only the core domain

Custom code belongs only in the distinctive scientific problem that justifies the package. That problem is the core domain. Linear algebra, optimization, numerical integration, automatic differentiation, parallel execution, file input and output, and plotting are generic capabilities: reuse mature libraries from the host stack instead of rebuilding them.

A package must document, test, and maintain everything it owns. Owning generic work therefore takes effort away from the scientific contribution, and a reimplement is usually less tested, less optimized, and less numerically robust than the established one. Implement the core; compose with the host stack for the rest.

2.2 Code is for life, not just for Christmas

Everything the package takes on is a commitment for the lifetime of the software, and that lifetime usually exceeds the involvement of the people who wrote it. Removing something later breaks the users who rely on it (Subsection 4.5). Judge each addition by what it will cost to maintain over years, not by the effort of adding it today, and prefer established, well-understood technology over the newest option.

Features. Every feature adds to what the package must document, test, and maintain (Subsection 2.1) for as long as it lives. Prefer a smaller package with well-tested core functionality over a larger one with many unfinished or poorly supported features.

Dependencies. Each dependency is code the package did not write but still has to install, pin, test against, license, and debug. Prefer the host stack the user already has, and add a dependency only when it owns a job the package must not reimplement and cannot leave to the caller. Choose dependencies that will outlive your own project: widely used, maintained by more than one person,

backed by an institution or an active community, and with a history of regular, compatible releases. Avoid dependencies that rest on a single maintainer, thesis, or grant; where a fragile one is unavoidable, isolate it behind a thin internal interface so that it can be replaced. Dependencies needed only for optional functionality, such as plotting, should be optional, so that the core installs without them.

The package itself. Build it to survive its authors. No part of the code should be understood by only one person, and the knowledge needed to maintain it, such as design decisions, conventions, and the release process, belongs in the repository, not in people's heads.

2.3 Don't call us, we'll call you

A framework follows the Hollywood principle: it owns the program and calls user code at fixed hooks. A library does the opposite: the user owns the program and calls the library. The scientific logic of a package should be library code.

Library code should not take over the main program, the solver or simulation loop, the data model, or the analysis workflow. Provide composable primitives that users call from their own scripts, notebooks, pipelines, and host-stack tools. The package should not become a framework on top of the host stack.

Where the method needs user code, such as an objective function, a model, or the right-hand side of a differential equation, accept it as an ordinary function argument. Requiring users to subclass package classes or to register code with a package-managed lifecycle turns the library into a framework.

A package that is itself a tool or a workflow does own its program, through its application layer (Subsection). Keep that layer thin: it orchestrates, handles files and configuration, and calls the library code, but contains no scientific logic. The science can then be tested, reused, and scaled without running the whole workflow.

2.4 Operate on data objects, not files

Library code should take data objects and return data objects; reading and writing files belongs to the caller or to the application layer (Subsection). This avoids assumptions about file formats, directory layouts, and environments, and it prevents hidden side effects.

Accept the data types of the host stack, including lazy, memory-mapped, and chunked representations where the method permits it. Operating on data objects does not mean loading everything into memory; materializing such data behind the user's back is a surprise cost (Subsection 5.6).

Keeping input and output out of library code creates an obligation: the package's own objects, such as fitted models or result containers, should be storable with the host stack's standard serialization mechanisms, or convertible to standard types that are. If reading a domain-specific file format is itself part of the core domain, isolate it in a separate module that returns standard data objects.

2.5 Avoid plotting in core functionality

Plotting is brittle, backend-dependent, and parameter-heavy, so core functionality should not plot. If the package provides plotting, strictly separate computation from visualization: functions that compute return results, and separate functions plot them.

Plotting functions should take computed results as input and never recompute them. Where the plotting library supports it, they should draw into a target the caller can pass in, return the plot object instead of displaying it, and leave global plotting styles untouched (Subsection 5.4). The plotting library should be an optional dependency (Subsection 2.2).

3 Code structure and abstraction

3.1 Derive abstractions from applications, not applications from abstractions

Build for concrete applications that someone actually wants to run. An abstraction invented first, in the hope that useful applications will follow from it, is a guess about demand, and such guesses fail in ways that become visible only once real work is attempted.

Generality that no application requested is speculative generality. It adds options, layers, and special cases that must be documented, tested, and maintained forever while solving no problem anyone has. The cost is paid immediately, the benefit remains hypothetical.

Several genuinely different use cases are usually required before the shared concept becomes visible; the rule of three sets the customary threshold at three. A single use case invites a design fitted too tightly to it, and no use case invites a design fitted to nothing.

Keep the driving applications in the repository as runnable examples or integration tests. They document intent, they fail when the design drifts away from practice, and they force every proposed feature to name the concrete task that justifies it. Software is a practical artifact and can only be judged in use.

3.2 Design modules around clear responsibilities

Organize code into modules with clear, coherent responsibilities. Each module should own one part of the package, such as validation, computation, preprocessing, or visualization, and its name should say which. Avoid miscellaneous helper modules such as `utils` when a more specific name would describe the purpose of the code; a module that cannot be named precisely usually has no single responsibility.

Dependencies between modules should point in one direction, from the specific to the fundamental. Computation must not depend on visualization, and core computation should not depend on input and output. Knowledge shared by several modules, such as validation checks and numerical defaults, belongs in one module that the others import (Subsection 3.3).

3.3 DRY: Don't repeat yourself

Every piece of knowledge should have a single, authoritative representation in the software. DRY is not primarily about code. Knowledge includes rules, assumptions, constants, numerical defaults, conventions such as axes and units, validation logic, data schemas, and the documentation that describes them.

When the same knowledge lives in several places, every change has to find and update all copies consistently. Copies that drift apart produce software that contradicts itself. Typical examples in scientific code are a tolerance hard-coded in several functions, the same input check reimplemented in each user-facing function, or a default value repeated in the function signature, its documentation, and the examples. Define such knowledge once and refer to it everywhere else. Where documentation has to state a value, generate it from the single definition if the tooling permits.

DRY concerns knowledge, not text. Code that looks different can still duplicate the same rule. Conversely, code that looks alike may encode different rules, and merging it is a mistake (see Subsection 3.5).

3.4 No magic numbers

Every numerical constant with a scientific or numerical role, such as a threshold, cutoff, tolerance, iteration limit, or physical constant, must have a name, a unit where it has one (Subsection 4.3), a single definition (Subsection 3.3), and a justification stated next to that definition: a reference, a derivation, or the reason for the choice. In scientific code, an unexplained 3.0 or 0.05 is often an unexamined modeling decision, and a reviewer cannot tell a deliberate choice from an arbitrary one.

If a constant affects the results and users might reasonably want a different value, expose it as a parameter; its default then becomes part of the API (Subsection 4.4). Take physical constants from an authoritative source in the host stack rather than typing them in by hand. Constants whose meaning is evident from the code, such as the 2 in 2π , need no name.

3.5 Duplicate code is cheaper than the wrong abstraction

Avoid premature abstraction. Similar-looking code should be unified only when it represents the same underlying concept or rule (Subsection 3.3). A wrong abstraction hides important differences, increases complexity, and makes future changes harder, because every change has to respect all of its callers at once. Prefer simple duplication until the shared idea is clear and stable.

A common sign of a wrong abstraction is a shared function that accumulates flags and parameters whose only purpose is to switch its behavior for particular callers. When that happens, inline the function back into its callers, remove from each copy what that caller does not need, and extract a new abstraction only once the genuinely shared rule has become visible.

4 API design

4.1 Write the API first

Write the code the user would write before writing the code that makes it work. The calling code is the part that must be good, because it is the part users see, learn, and depend on. Everything behind the interface can be rewritten later; the interface itself cannot.

Designing outward from the implementation produces signatures that expose internal structure rather than user intent. Designing inward from the call site produces the opposite, and reveals awkwardness while it is still free to fix.

Judge a proposed interface by reading realistic usage rather than the definition. Argument order, naming, and defaults that appear reasonable in a signature often read badly in the script that calls them, and that script is the true specification.

An interface that cannot be written cleanly is a signal that the underlying concept is not yet clear. Resolve that before implementing, since implementation effort tends to entrench whatever interface it was built to serve.

4.2 Make the common case easy, and the uncommon case possible

A useful design principle for scientific software is to provide the API in *layers of abstraction*. Each layer exposes the same underlying capabilities but with different trade-offs between convenience and control.

Higher layers provide functions that solve common tasks with minimal user effort. These functions often make reasonable assumptions, provide sensible defaults, and combine multiple lower-level operations into a single call. They are designed for the typical use case.

Lower layers expose smaller, more specialized primitives. These functions require more explicit code from the user but provide greater flexibility, composability, and control over the computational workflow.

Each layer should be built from the public primitives of the layer below, so that the convenient functions are compositions of the controllable ones rather than a second implementation of the same knowledge (Subsection 3.3).

The relationship between abstraction layers, control, and convenience is illustrated in Figure 1. As abstraction layers increase, convenience typically increases while direct control decreases.

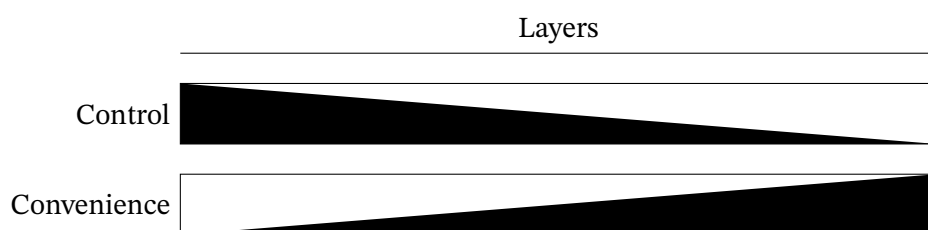


Figure 1: Layered API design. With increasing abstraction layers, direct control decreases while convenience increases.

4.3 Name axes, units, and conventions

Axis order, such as observations-by-variables versus variables-by-observations or time-major versus space-major, the index base, coordinate systems and reference frames, angle conventions, and physical units are part of the meaning of every array. If they are not in the signature, the documentation, or the returned object, users will infer them wrongly. The package must not infer them from the data either (Subsection 5.2).

Do not rely on a comment or an example notebook to carry a convention. Name the axes, state the units, and prefer the labeled data structures of the host stack where they exist. Keep identifiers the caller supplied, such as sample IDs, gene names, station codes, or node labels, intact and aligned with the data through every operation.

4.4 Numerics and defaults are part of the API

Changing a default normalization, initialization, regularization strength, convergence tolerance, iteration limit, ϵ , reduction, or tie-breaking rule can change published numbers. Such changes must be treated as breaking changes of the API, not as internal tweaks.

Signatures are not the whole contract. Documented numerical defaults are part of the scientific behavior of the package. If a default has to change, follow the process for breaking changes in Subsection 4.5.

4.5 Design stable APIs

API stability builds user trust, and in science it protects published results. Avoid breaking changes that are not necessary. Keep the public API small, since everything public is a commitment, and mark internal code as private using the host stack's convention.

Number releases so that the version communicates compatibility, for example with semantic versioning, and record every change of behavior in a changelog. Changes of numerical behavior, including changes of defaults (Subsection 4.4), must be listed explicitly in the changelog, not summarized as fixes.

When a breaking change is necessary, deprecate before removing: emit a deprecation warning (Subsection 5.8) that names the replacement, keep the old behavior available for a stated period, and document the migration path.

4.6 Document the method, not the implementation

Documentation should describe what a function computes and how to use it, not how the code is organized internally. Clear interfaces allow the implementation to change without breaking user code.

In scientific software, however, the method is part of the behavior. The documentation of a user-facing function should state the algorithm or estimator, the approximations and assumptions it makes, its

numerical defaults, its policy for missing values (Subsection 6.3), and references to the literature that defines the method. Users need this knowledge to interpret and report their results.

Implementation details, such as loop structure, internal data structures, and private helper functions, do not belong in user documentation. The dividing line is whether a change could alter a scientific result: if it could, it is part of the method and must be documented; if it could not, it is implementation.

5 Explicit behavior and no side effects

5.1 Do what it says on the tin

A function should do what its name and arguments say, and nothing more. Processing steps that change the data, such as filtering, normalization, sorting, deduplication, or outlier removal, must not be hidden inside a function named for a different operation: a user who calls a fitting function should not discover later that the data were also filtered. Expose such steps as named arguments, or provide them as separate functions the user calls. Steps that are part of the definition of the method are documented as such (Subsection 4.6).

This is the counterpart of Subsection 5.2: a function must neither read more into a call than it states nor do more than it asks.

5.2 Don't guess

The package must not resolve an ambiguity in the user's intent on the user's behalf. A default is not a guess: it is fixed, documented, and independent of the data. A guess applies heuristics to the data to decide what the call means, such as transposing a matrix because of its shape, detecting whether data are already normalized, inferring units or date formats from the values, matching labels approximately, or repairing malformed input. If such an interpretation could change the result, the user has to decide.

Make the user state the choice through an argument, or raise an error that lists the possible interpretations. Where automatic selection is genuinely useful, offer it as an explicit option, such as a method argument with the value `auto`, and record the choice in the returned result. Heuristics may trigger warnings (Subsection 5.8), but must not change behavior.

5.3 Do not mutate caller data

A function must not overwrite, transpose, densify, drop missing values from, or otherwise rewrite objects the caller still holds, unless the user opted into an explicit in-place operation, such as a clearly named in-place function or argument. Silent mutation makes it impossible to know which object still holds the original data.

If a transformation is required, return a new object. The default is: leave caller data untouched.

5.4 Do not touch global state

Library code must not change process-wide settings. It must not set or reseed the global random number generator, select hardware devices, change the number of threads used by numerical libraries, change floating-point error handling, change display options of data libraries, or rewrite plotting defaults. These settings belong to the user. A package that calls `set.seed` in R or `numpy.random.seed` in Python inside a function silently takes control of the user's session.

A change that is strictly scoped to the package's own computation, and restored before control returns to the caller, including when an error occurs, may be made where the host stack provides a mechanism for it. How the caller controls randomness instead is defined in Subsection 8.4.

5.5 The library follows the caller's data types

If the caller passes data of a particular type, precision, or placement, keep it. Library code must not silently densify sparse data, upcast single to double precision, move data between the main processor and an accelerator, or convert one array type of the host stack into another. The output should have the same kind of type, precision, and placement as the input. Inputs that carry no such choice, such as a plain list of numbers, may be converted to the package's working type.

The caller's choices reflect deliberate trade-offs of memory, speed, precision, and hardware. Following the incoming data preserves them; choosing for the user breaks the workflows built on them.

Correctness outranks this rule (Subsection 1.2). If the method cannot produce a correct result in the incoming precision, raise an error that names the required precision, rather than silently upcasting or silently returning a degraded result; a user who wants the conversion can make it explicitly. Using higher precision for small intermediate quantities, such as an accumulator, is part of the method and must be documented as such (Subsection 4.6); converting the caller's data is not. If the package cannot operate on a given type at all, it should say so with an error rather than convert the data at hidden cost.

5.6 No surprise cost

Time, memory, and network use are part of the contract. A helper must not, as a side effect, densify sparse data, materialize lazy data, download data, or transfer a whole array to or from an accelerator.

Expensive steps should be explicit in the function the user calls, or impossible to invoke by accident. A convenience wrapper that silently materializes the full dataset is a bug, not a feature. Input validation has costs as well; how they are balanced against correctness is defined in Subsection 6.2.

5.7 Importing the package has no side effects

Importing or loading the package must not download data, compile extensions, print, select backends, configure logging, or read from or write to the user's home directory. Users load libraries inside other tools, test suites, and notebooks; work done at load time is hidden work, and it slows down and destabilizes everything that depends on the package.

Put optional heavy setup behind an explicit function the caller invokes. Loading the package should be cheap and silent.

5.8 Raise, warn, or stay silent

Library code must not print. Standard output belongs to the user's scripts, notebooks, and batch jobs. The package communicates through three channels, and each situation calls for exactly one of them:

- **Raise** when the call is invalid, or when continuing would return an untrustworthy result as if it were valid. The user has to act, and execution must not continue as if nothing happened.
- **Warn** when the call is valid but is probably not what the user intended, for example when a heuristic suggests that the data are not what the call assumes (Subsection 5.2), or when the behavior of the call will change in a future version. Deprecations (Subsection 4.5) belong here. Use the host stack's standard warning mechanism, so that users can filter warnings or turn them into errors.
- **Stay silent** when the call is valid and unremarkable.

A warning must not replace an error: if continuing would produce a wrong result, raise. Warnings should name the argument or condition that triggered them and say how to avoid them by changing the call. Avoid emitting the same warning repeatedly, for example inside a loop.

Progress reports and diagnostic messages fall outside these three channels. They should be opt-in, through an explicit argument or the host stack’s standard logging or messaging facility, so that the user decides whether and where they appear.

6 Input validation and errors

6.1 Garbage in, useful error out

User-facing functions must validate their inputs at the point of entry: types, shapes, bounds, units where they are declared, and the semantic constraints of the method. Invalid inputs, unsupported combinations of arguments, and dangerous settings must produce an error at the boundary, not a numerical failure or a wrong result further downstream. Fail loudly and early.

Error messages should be specific enough that users can recover without searching through the full documentation: name the argument, state what was expected and what was received, and where possible say how to fix the call.

Do not rely on documentation for correctness. Many users read documentation only partially, skim examples, or rely on auto-completion and error messages while using the package. Documentation explains correct usage; the code must enforce the constraints that correctness depends on. Make misuse difficult and failures obvious.

Implement each check once, in the validation module (Subsection 3.2), and call it from every entry point that needs it (Subsection 3.3). Private functions that are reached only through a validated entry point should not repeat the checks.

6.2 Keep validation proportionate

Validation has a cost, so the requirements for correctness and for cost (Subsection 5.6) can pull in different directions. Distinguish two kinds of check:

Metadata checks inspect only types, shapes, precision, declared units, and combinations of arguments. They are cheap and must always run in user-facing functions.

Data checks inspect the values themselves, for example for missing values, bounds, symmetry, or sortedness. They cost at least one pass over the data, and for data on an accelerator they can force the program to wait for the device.

Correctness outranks performance (Subsection 1.2), so data checks should run by default in user-facing functions, and their cost should be documented. A user who has already validated the data may turn them off through an explicit, clearly named argument. This follows the layered design of Subsection 4.2: the common case is safe, and the uncommon case is possible.

Data checks must not themselves create a surprise cost (Subsection 5.6). For lazy or chunked data, express a check as part of the same deferred computation, or apply it chunk by chunk.

6.3 Missing values and NaN are explicit

Each user-facing function must have a documented policy for missing values, such as NaN, NA, or masked entries: reject, propagate, or mask. Silent imputation, dropping, or filling is a scientific bug, not a convenience: it replaces a missing measurement with a guess (Subsection 5.2).

If the input violates the policy, the function must raise an error that names the missing values, rather than fail later with an unrelated numerical error. Checking for missing values is a data check in the sense of Subsection 6.2.

7 Numerical correctness

7.1 Prefer numerically stable formulations

A formula that is correct in exact arithmetic can be badly wrong in floating point. Choose formulations that remain accurate for the whole range of inputs the package accepts, not only for well-behaved test data. Common patterns include:

- Avoid subtracting nearly equal quantities, which cancels significant digits. Rearrange the formula, or use dedicated functions such as `log1p`, `expm1`, and `hypot`.
- Work in log space for products of many small numbers, such as likelihoods, and compute $\log \sum_i e^{x_i}$ as $m + \log \sum_i e^{x_i - m}$ with $m = \max_i x_i$.
- Solve linear systems instead of forming explicit matrix inverses, and prefer factorizations suited to the structure of the problem.
- Accumulate long sums with care, for example by pairwise or compensated summation, or with a higher-precision accumulator (Subsection 5.5).

Reuse the stable routines of mature libraries rather than reimplementing them (Subsection 2.1). Test behavior at the boundaries of the input range, such as very small and very large magnitudes and nearly singular inputs, as part of the invariant tests of Subsection 8.2.

7.2 Report conditioning and convergence

Distinguish an unstable algorithm from an ill-conditioned problem. An unstable algorithm is a defect of the package and must be fixed. An ill-conditioned problem cannot be solved accurately by any algorithm; the package cannot fix it, but it must not hide it. Where conditioning can be estimated at reasonable cost, check it and report an ill-conditioned problem through the channels of Subsection 5.8.

Iterative methods must not return a result that did not converge as if it had. When the iteration limit is reached or the convergence criterion fails, either raise, or return a result that carries an explicit convergence status and warn. Each iterative function must document which of the two it does, and its result should record diagnostics such as the number of iterations and the final residual.

8 Testing and reproducibility

8.1 Testing is not optional

Scientific code is particularly prone to subtle errors that produce plausible but wrong numbers. Every user-facing function must have tests, and every release must pass the full test suite.

Tests should cover error paths as well as results: invalid inputs must be shown to raise the documented errors (Subsection 6.1). Every fixed bug should gain a regression test that fails without the fix. Test the range of input types the package claims to support, such as sparse, single-precision, or lazy data (Subsection 5.5); a code path that is never exercised is not supported.

Run the test suite automatically on every change, across the versions of the language and of the host-stack libraries that the package supports.

8.2 Test invariants, not only examples

Passing on a few happy-path inputs is a weak scientific test. Prefer invariants: conservation laws, symmetries, known analytic solutions and limiting cases, consistency relations such as counts and normalizations, and agreement with an independent implementation.

Examples are useful, but they do not prove that the method is correct. Encode the scientific claims the software makes and test those claims directly.

8.3 Size matters

A method that works on test data but fails at the scale of real applications solves no problem. Document the time and memory complexity of each user-facing function in terms of the data size, and benchmark at the sizes of the driving applications (Subsection 3.1), not only on toy inputs. Profile before optimizing, since bottlenecks are rarely where intuition expects them, and track benchmarks to catch performance regressions. Performance never outranks correctness or clarity (Subsection 1.2).

8.4 Reproducibility: the same answer under stated conditions

Results must be reproducible. In these guidelines, reproducibility means obtaining the same scientific answer, within a stated tolerance, from the same inputs, parameters, and package version, on a documented software and hardware stack. It does not mean identical bits forever: floating-point results legitimately vary across linear algebra libraries, processors, accelerators, and compiler settings.

Randomness must be controllable by the caller. Where the host stack supports it, every function that uses random numbers should accept a random number generator or seed argument that controls all randomness within that call. Where the host stack's convention is a single global generator, draw from the generator the caller controls, subject to Subsection 5.4. The behavior when no generator or seed is supplied must be documented.

Prefer deterministic algorithms. Where nondeterminism is unavoidable, for example because parallel reductions sum in varying order, document it and ensure that results still agree within the stated tolerance.

8.5 Compare floats with tolerance

Tests should compare floating-point results within a tolerance, not bit for bit. Bit-identical reference files treat platform noise as a logic bug and make the test suite fragile.

Assert agreement within the tolerance that defines reproducibility (Subsection 8.4). Use relative tolerances for quantities of varying magnitude and absolute tolerances for quantities near zero. Tolerances are constants in the sense of Subsection 3.4: derive each from the expected numerical error of the method, rather than adjusting it until the tests pass on one machine.

8.6 Flaky tests cry wolf

A test that fails at random teaches people to ignore failures, and a test suite whose failures are ignored checks nothing. Tests of stochastic methods must therefore fix the random generator (Subsection 8.4), so that every run gives the same outcome.

A fixed generator makes a test repeatable, not convincing: a test that passes for only one seed can hide a bug. Where the claim is statistical, such as an unbiased estimate or a coverage rate, assert it with bounds whose false-failure rate is known and very small, for example a deviation of several standard errors, so that the test would pass for other seeds as well. The bound is a constant in the sense of Subsection 3.4. Never choose a seed because the test passes with it.

8.7 Make every result traceable to a version

A result is reproducible only if the exact software that produced it can be identified and obtained again, whether the result ends up in a publication, a report, or a decision.

The package must expose its version at run time through the host stack's standard mechanism, such as `__version__` in Python or `packageVersion` in R. Every release should be tagged in version control and archived so that the exact version can be retrieved later: in a public archive with a persistent

identifier, such as a DOI, for published software, or in the organization's release archive for internal software. Published software should also provide machine-readable citation metadata, such as a `CITATION.cff` file, stating how to cite both the software and the method.

Results should record the package version and the parameters that produced them, including defaults that were not set explicitly: library code records them in the result objects it returns, and the application layer records them in the outputs and run metadata it writes. Changes of behavior between versions are traced through the changelog (Subsection [4.5](#)).